

CAN Bus Saldırı Tespitinde Chebyshev Spektral Graf Sinir Ağları

Tuna Girişken

Öğrenci No: 91250000319

Bilgisayar Mühendisliği Bölümü

Ege Üniversitesi, Fen Bilimleri Enstitüsü

Ders: Çizge Teorisinde Ölçüm Parametreleri

35100 İzmir, Türkiye

Özet

Modern araçlarda elektronik birimler (ECU) Controller Area Network (CAN) bus üzerinden haberleşir; bu yapı hızlı ve güvenilir olsa da mesajların kaynağını doğrulamaz. Kötü niyetli trafik veya sahte komutlar (DoS, Fuzzy, RPM/Gear spoofing) sürüş güvenliğini tehdit edebilir. Bu rapor, araç içi ağlarda saldırı tespiti için ChebNet adlı spektral graf sinir ağı modelinin KAIST veri kümesi üzerinde nasıl uygulandığını adım adım açıklar.

KAIST kayıtlarında her 100 ms'lik CAN trafiği bir *graf* olarak modellenir: benzersiz mesaj kimlikleri (CAN ID) düğüm, ardışık frame geçişleri kenar olarak temsil edilir; her graf tek bir etiket alır (Normal, DoS, Fuzzy, RPM Spoof veya Gear Spoof). ChebNet, bu çizgeler üzerinde komşuluk yapısını spektral filtrelerle öğrenerek her segmentin saldırı türünü tahmin eder. Rapor; veri yükleme ve segmentleme, model mimarisi, eğitim ve değerlendirme adımları ile üretilen metrik, tablo ve şekilleri bir arada sunar.

Tam KAIST önbelleği üzerinde (yaklaşık 111 000 graf, 10 tur eğitim, %70 eğitim / %30 doğrulama bölmesi) elde edilen sonuçlarda makro düzeyde F1-skoru 0,956 ve AUC 0,998 ölçülmüştür: Normal, DoS ve Fuzzy sınıfları yüksek doğrulukla ayrılırken RPM ve Gear spoof türleri görece daha zor ayırt edilmiştir. Veri akışı şeması, örnek segment görseli, karışıklık matrisi ve ROC eğrileri sürecin hem teknik hem yorumlanabilir yönünü okuyucuya aktarır.

Anahtar Kelimeler

CAN bus, saldırı tespiti, graf sinir ağı, ChebNet, spektral filtreleme, otomotiv siber güvenliği, graf sınıflandırma

I. GİRİŞ

Bu rapor, Ege Üniversitesi Fen Bilimleri Enstitüsü Bilgisayar Mühendisliği Bölümü *Çizge Teorisinde Ölçüm Parametreleri*

treleri dersi kapsamında hazırlanmıştır. Çalışma, KAIST CAN bus saldırı tespiti veri kümesinde ChebNet modelinin eğitim ve değerlendirme sürecini belgelemeyi amaçlar.

Araç içi CAN haberleşmesi güvenlik açısından kritik olmasına rağmen doğal olarak mesaj seviyesinde kimlik doğrulama sağlamaz [1]. Bu nedenle DoS, Fuzzy ve spoofing gibi saldırıları erken yakalayabilen IDS yaklaşımları gereklidir. İletişim ve araç ağlarında graf yapısını doğrudan modelleyen GNN yaklaşımları bu bağlamda giderek daha fazla ilgi görmektedir [2]–[4].

Bu çalışmada görev, KAIST kayıtları [5] üzerinde graf düzeyinde sınıflandırma olarak tanımlanır: her 100 ms CAN segmenti bir graf olarak ele alınır ve beş sınıftan birine atanır (Normal, DoS, Fuzzy, RPM Spoof, Gear Spoof). ChebNet, bu graf temsili üzerinde komşuluk yapısını öğrenerek segment düzeyinde saldırı türü tahmini yapar [6]. Dikkat tabanlı GAT mimarileri kenar ağırlıklarını öğrenirken [7], ChebNet spektral komşuluk filtreleriyle yapısal desenleri yakalar.

A. Çalışmanın Çıktıları

- KAIST tam veri kümesi üzerinde ChebNet eğitim ve değerlendirme süreci.
- Sınıf bazlı performans tabloları, karışıklık matrisi ve değerlendirme şekilleri.

II. KAVRAMSAL ÇERÇEVE

A. CAN Trafik Verisinin Graf Temsili

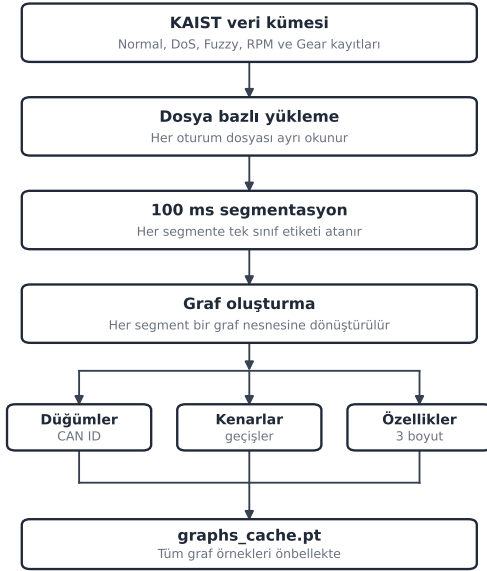
Her segmentte benzersiz CAN ID'leri düğüm, ardışık CAN frame geçişleri yönlü kenar olarak tanımlanır (aynı CAN ID art arda gelirse kenar oluşturulmaz). Düğüm özellikleri üç boyuttan oluşur: log-frekans, normalize DLC ortalaması ve ID öncelik bilgisi. Bu üç özellik, KAIST segmentlerinden üretilen tüm ChebNet graf örneklerinde aynı biçimde kullanılır.

B. KAIST Verisinin Hazırlanması

KAIST veri kümesinde beş saldırı türü ayrı kayıt dosyaları halinde sunulur (normal, DoS, Fuzzy, RPM ve Gear). Kayıtlar farklı oturumlardan geldiği için zaman damgaları dosyalar arasında sürekli değildir; bu nedenle tüm dosyalar tek bir zaman çizelgesinde birleştirilmez. Bunun yerine her dosya ayrı işlenir ve elde edilen graf örnekleri eğitim havuzunda bir araya getirilir.

Veri hazırlama adımları özetle şunlardır:

- 1) Her kayıt dosyası ayrı yüklenir ve alan tipleri standartlaştırılır.
- 2) Her dosya 100 ms'lik segmentlere ayrılır (segmentasyon).
- 3) Her segment bir graf örneğine dönüştürülür.
- 4) Tüm graf örnekleri eğitim ve değerlendirme için havuzlanır.



Şekil 1: KAIST veri kümesinden graf önbelleğine veri hazırlama akışı.

Bu yaklaşım, her örneğin yaklaşık aynı süreyi temsil etmesini sağlar. Böylece model, trafik yoğunluğu değişse bile karşılaştırılabilir zaman dilimlerinden öğrenir.

C. ChebNet Modelinin Temel İlkesi

ChebNet (Chebyshev Network), graf sinir ağlarının spektral ailesine aittir ve 2016'da Defferrard ve ark. [6] tarafından önerilmiştir. Klasik evrişimli sinir ağları düzenli ızgara

(görüntü pikseli) üzerinde çalışır; ChebNet ise düzensiz graf yapısında—burada CAN kimlikleri ve aralarındaki geçişler—öğrenme yapar. Temel fikir, her düğümün özelliğini yalnızca doğrudan komşularından değil, graf üzerinde birkaç adım ötedeki düğümlerden gelen bilgiyle birlikte güncellemektir.

Bunu sağlayan yapı taşı ChebConv katmanıdır. Katman, graf Laplacian'ı üzerinde Chebyshev polinomlarıyla tanımlanan spektral filtreler uygular. Düz bir dille: model hem *yakın* komşuluk desenlerini (ör. ardışık iki CAN ID arasındaki sık geçiş) hem de *daha geniş* bağlamı (birkaç atlama sonrası ulaşılabilen düğüm çevresi) aynı anda dikkate alabilir. Bu çalışmada polinom derecesi $K = 3$ seçilmiştir; yani her katmanda yaklaşık üç atlama komşuluğuna kadar bilgi toplanır.

ChebNet'in CAN saldırı tespiti için uygun olmasının nedeni, saldırıların çoğu zaman yalnızca tek bir mesaj kimliğini değil, mesajlar arası *geçiş düzenini* bozmasıdır. DoS trafiği farklı bir frekans ve geçiş yoğunluğu üretir; spoof saldırıları belirli kimliklerin normal trafikle birlikte nasıl görüldüğünü değiştirir. Spektral filtreler, bu tür yapısal farkları düğüm özellikleriyle birleştirerek segment düzeyinde ayırt edici bir temsil öğrenmeye yardımcı olur.

Bu projede kullanılan ChebNetIDS mimarisi üç ChebConv katmanından oluşur; her katmanda ELU aktivasyonu uygulanır. Katmanlar sırasıyla $3 \rightarrow 64$, $64 \rightarrow 64$ ve $64 \rightarrow 128$ boyutuna dönüştürme yapar ($K = 3$). Düğüm temsilleri `global_mean_pool` ile graf düzeyinde tek bir vektöre indirgenir; ardından iki katmanlı bir sınıflandırıcı başlık beş saldırı sınıfı için logit üretir. Görev graf düzeyindedir: her 100 ms segment bir graf olarak verilir ve model o segmentin tamamı için tek bir etiket tahmin eder (Normal, DoS, Fuzzy, RPM Spoof veya Gear Spoof).

III. METODOLOJİ VE DENEY SÜRECİ

A. Segmentasyon ve Etiketleme

Her kayıt 100 ms ($\Delta t = 0.1$ s) segmentlere ayrılır. Segmentte saldırı çerçevesi yoksa etiket Normal (0) kabul edilir; varsa saldırı çerçevelerinin çoğunluk etiketi o segmentin sınıfı olur. Böylece her segment tek bir graf ve tek bir sınıf etiketiyle temsil edilir.

B. Graf Yapısının Türetilmesi

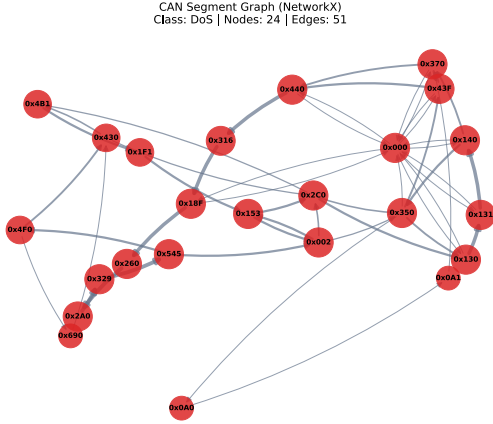
Her segmentte:

- **Düğüm**ler o segmentte görülen benzersiz CAN kimlikleridir.
- **Kenarlar** art arda gelen farklı CAN kimlikleri arasındaki geçişlerdir.

- **Düğüm özellikleri** frekans, ortalama veri uzunluğu ve kimlik önceliğinden oluşur.
- **Kenar ağırlığı** aynı geçişin segment içindeki tekrar sayısıdır.

Tek düğümlü veya kenarsız örneklerde hesaplamaların kesintisiz sürmesi için yapay bir geri dönüş kenarı eklenir; bu yalnızca teknik bir önlemdir ve etiketi değiştirmez.

C. Örnek Segment Görselleştirmesi



Şekil 2: KAIST verisinden bir saldırı segmentinin graf temsili. Düğüm boyutu frekans özelliğiyle, kenar kalınlığı geçiş tekrarıyla orantılıdır.

Gerçek bir KAIST saldırı segmenti Şekil 2’de gösterilmiştir. Düğümler CAN kimliklerini, kenarlar mesajlar arası geçişleri temsil eder. Bu görsel, soyut graf temsilinin somut bir örneğini sunar; eğitim sürecinin zorunlu bir parçası değildir.

D. Eğitim ve Değerlendirme Düzeni

Segmentasyon ve graf oluşturma tamamlandığında tüm örnekler `data/kaist/graphs_cache.pt` dosyasına kaydedilir. Her kayıt bir grafı temsil eder: düğüm özellikleri, kenarlar ve segment etiketi. Eğitim bu önbellekten okur; ham CAN dosyaları eğitim sırasında yeniden işlenmez.

Graf örnekleri %70 eğitim ve %30 doğrulama olacak şekilde bölünür; her saldırı türünden her iki kümede de benzer oranda örnek kalması için sınıf dağılımı korunur (seed 42). Eğitimde her adımda 64 graf birlikte modele verilir; ChebNet her graf için ayrı bir sınıf tahmini üretir.

Az örnekli sınıflar eğitimde daha sık seçilir; böylece model yalnızca baskın sınıflara uyum sağlamaz. Model, eğitim kümesindeki tüm grafları 10 kez görecektir; her tur sonunda doğrulama kümesindeki performans ölçülür.

En yüksek makro-F1 skoruna sahip model `checkpoints/best_model_chebnet.pth` dosyasına kaydedilir.

Değerlendirme aşamasında aynı bölme ve kaydedilen model kullanılır; metrikler `-full-report` ile üretilir.

E. Yazılım Ortamı

ChebNet deney yazılım yığını			
Çalışma ortamı	Python 3.10	YAML config	gnn_canids CLI
Veri	pandas	NumPy	graphs_cache.pt
Model	PyTorch	PyG	ChebConv
Değerlendirme	scikit-learn	makro metrikler	train/val bölmesi
Görselleştirme	matplotlib	seaborn	NetworkX
Çıktı	best_model.pt	metrik tabloları	rapor şekilleri

Şekil 3: Yazılım katmanları: yapılandırmadan model, metrik ve rapor çıktılarına.

Deneyler Python 3.10 ortamında yürütülmüştür. Model ve eğitim döngüsü PyTorch üzerine kuruludur; ChebNet katmanları PyTorch Geometric kütüphanesindeki ChebConv bileşeniyle sağlanır [8]. Ham KAIST kayıtlarının okunması, segmentasyonu ve graf önbelleğinin oluşturulması pandas ve NumPy ile yapılır; eğitim/doğrulama bölmesi ile makro metrikler scikit-learn yardımcıları kullanılarak hesaplanır. Hiperparametreler ve veri yolları YAML yapılandırma dosyalarında (`configs/train_chebnet.yaml`) tutulur; böylece eğitim ve değerlendirme komutları aynı ayarlarla tekrarlanabilir. Tam rapor üretiminde matplotlib ve seaborn ile sınıflandırma şekilleri oluşturulur; örnek segment görseli için NetworkX kullanılmıştır. Şekil 3 bu bileşenlerin deney akışındaki yerini özetler.

IV. DENEY BULGULARI

Deney, KAIST tam veri önbelleğindeki yaklaşık 111 000 graf üzerinde yürütülmüştür. Örnekler %70 eğitim ve %30 doğrulama ($n = 33\,381$) olacak şekilde bölünmüş; model 10 tur boyunca batch boyutu 64 ile eğitilmiştir. Eğitim ve değerlendirme sırasıyla aşağıdaki komutlarla çalıştırılmıştır:

- `python3 -m gnn_canids.cli.train -config configs/train_chebnet.yaml`
- `NUMBA_CACHE_DIR=. numba_cache python3 -m gnn_canids.cli.evaluate -model chebnet`

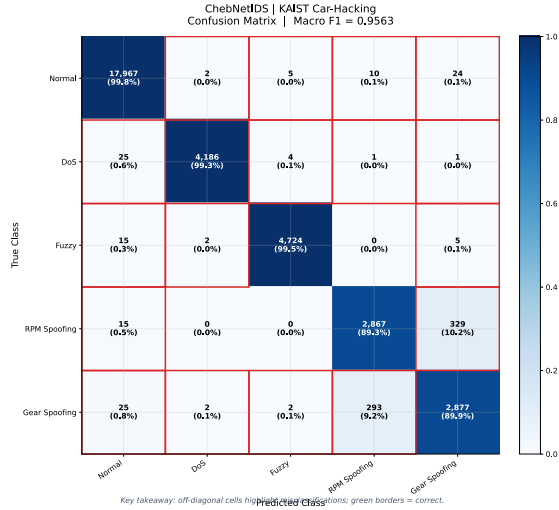
```
-config configs/train_chebnet.yaml
-full-report
```

En iyi doğrulama performansı 10. turda elde edilmiştir: makro F1 **0,9563**, makro AUC **0,9977**, makro precision 0,9570 ve makro recall 0,9556. Tablo I bu sonuçları sınıf bazında ayrıştırır; Normal, DoS ve Fuzzy sınıfları 0,996–0,997 F1 aralığında iken RPM ve Gear spoof sırasıyla 0,899 ve 0,894 F1 ile en zor ayırt edilen kategorilerdir.

Tablo I: Sınıf bazlı metrikler (ChebNet tam KAIST, 10 tur eğitim)

Sınıf	Precision	Recall	F1
Normal	0.996	0.998	0.997
DoS	0.999	0.993	0.996
Fuzzy	0.998	0.995	0.997
RPM Spoof	0.904	0.893	0.899
Gear Spoof	0.889	0.899	0.894
Makro Ort.	0.957	0.956	0.956

A. Karışıklık Matrisi

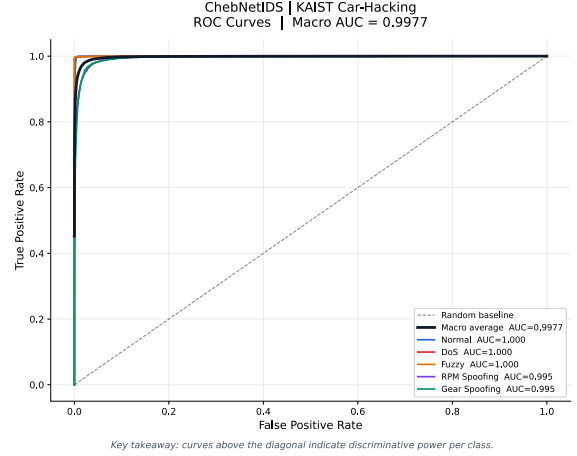


Şekil 4: ChebNet KAIST tam veri analizi (10 tur) karışıklık matrisi.

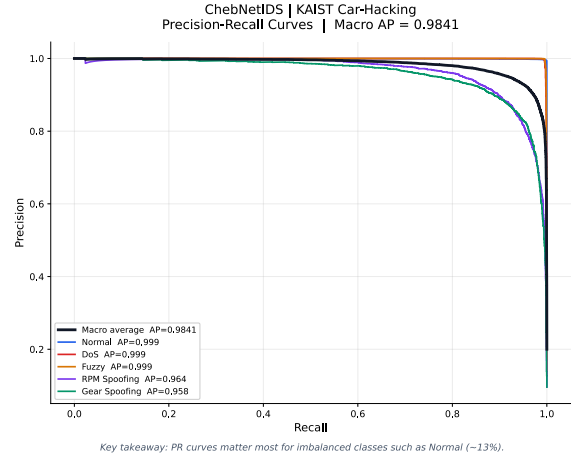
Tablo I makro ortalama F1 değerini 0,956 olarak özetler; RPM ve Gear spoof sınıfları makro metrikleri sınırlayan ana kaynaktır. Şekil 4 bu tabloyu görsel olarak tamamlar.

Karışıklık matrisi, Tablo I’de görülen RPM ve Gear spoof zayıflığını doğrular: kalan hataların çoğu bu sınıflar ile Normal veya birbirleri arasında yoğunlaşır.

B. ROC ve Precision-Recall Eğrileri



Şekil 5: ChebNet KAIST tam veri analizinde (10 tur) beş sınıf için one-vs-rest ROC eğrileri.



Şekil 6: Precision-recall eğrileri; az temsil edilen sınıflarda risk daha net görülür.

ROC eğrileri tüm sınıflarda güçlü ayrım gösterir. Precision-recall eğrileri ise özellikle RPM ve Gear spoof gibi az temsil edilen sınıflarda gerçekçi bir risk tablosu sunar: AUC yüksek olsa bile sabit karar eşiğinde yanlış alarm veya kaçırılan saldırı oranı yüksek kalabilir.

V. BULGULARIN YORUMLANMASI

Sonuçlar, ChebNet’in KAIST tam veri kümesinde güçlü bir sınıflandırıcı olduğunu gösterir. Normal, DoS ve Fuzzy sınıfları neredeyse tam ayrılmış; RPM ve Gear spoof sınıfları makro metrikleri sınırlayan ana kaynaktır. Rastgele bölme, aynı kayıt oturumundan gelen örnekler arasında benzerlik taşıyabileceğinden metrikler iyimser yorumlanmalıdır.

ChebNet, KAIST üzerinde yüksek makro-F1 sunmuştur. Spoof sınıflarındaki düşük skorlar, normal trafikle kısmi örtüşme ve sınıf dengesizliğiyle ilişkilidir; ROC eğrileri güçlü ayırım gösterirken precision-recall eğrileri bu sınıflarda daha temkinli bir tablo ortaya koymaktadır. GNN çıktılarının yorumlanabilirliği konusunda literatürde kapsamlı çalışmalar bulunmaktadır [9].

VI. SONUÇ

Bu rapor, araç içi CAN bus trafiğinde saldırı tespiti için ChebNet tabanlı bir graf sınıflandırma yaklaşımını KAIST veri kümesi üzerinde uçtan uca belgelemektedir. Çalışmanın temelinde her kayıt dosyasının 100 ms'lik segmentlere ayrılması, her segmentten CAN kimlikleri ve ardışık geçişlerden oluşan bir grafın türetilmesi ve bu grafların ChebNet ile beş saldırı sınıfına (Normal, DoS, Fuzzy, RPM Spoof, Gear Spoof) atanması yer almaktadır. Böylece ham mesaj akışı, çizge yapısına dönüştürülerek spektral graf sinir ağı ile işlenebilir bir biçimde sunulmuştur.

Tam KAIST önbelleği üzerinde yaklaşık 111.000 graf ile yürütülen deneyde, %70 eğitim / %30 doğrulama bölmesi ve 10 tur eğitim sonrasında model güçlü bir genel performans göstermiştir. Doğrulama kümesinde makro-F1 0,956 ve makro-AUC 0,998 değerleri elde edilmiştir. Normal, DoS ve Fuzzy sınıflarında F1 skorları 0,996–0,997 aralığında seyrederken; RPM ve Gear spoof sınıfları sırasıyla 0,899 ve 0,894 F1 ile en zor ayırt edilen kategoriler olmuştur. Bu tablo, ChebNet'in yoğunluk ve geçiş desenindeki belirgin değişiklikleri (DoS, Fuzzy) güvenilir biçimde yakalayabildiğini; ancak normal trafikle kısmen örtüşen spoof türlerinde daha fazla karışıklık ürettiğini göstermektedir.

Karışıklık matrisi (Şekil 4) ile ROC ve precision-recall eğrileri (Şekil 5 ve Şekil 6) Tablo I'deki bulguları görsel olarak desteklemiştir. Özellikle spoof sınıflarında yüksek AUC'ye rağmen precision-recall dengesinin daha temkinli kalması, sınıf dengesizliğinin operasyonel yorumda dikkate alınması gerektiğini hatırlatmaktadır. Veri akışı şeması (Şekil 1) ve örnek segment görseli (Şekil 2) ise soyut pipeline'ın somut bir CAN grafına nasıl indirgendiğini okuyucuya aktarmıştır.

Çalışma tek araç veri kümesi, tek rastgele tohum ve sabit $K = 3$ seçimiyle sınırlandığından sonuçlar dikkatle yorumlanmalıdır; buna rağmen elde edilen metrikler, ChebNet'in KAIST CAN saldırı tespiti görevinde uygulanabilir bir çözüm sunduğunu ortaya koymaktadır. Bu rapor, veri hazırlamadan değerlendirme çıktılarına kadar tüm süreci kayıt altına alarak çalışmayı tamamlamaktadır.

- [1] N. Choudhury, R. Bestak, and L. Hanus, "Intrusion detection systems for networked control systems: A survey," *IEEE Access*, vol. 9, pp. 51 543–51 571, 2021.
- [2] J. Suárez-Varela, P. Almasan, M. Ferriol-Galmés, K. Rusek, F. Geyer, X. Cheng, X. Shi, S. Xiao, F. Scarselli, A. Cabellos-Aparicio, and P. Barlet-Ros, "Graph neural networks for communication networks: Context, use cases and opportunities," *IEEE Network*, vol. 37, no. 3, pp. 140–147, 2023.
- [3] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2022, pp. 1–9.
- [4] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, "Anomal-E: A self-supervised network intrusion detection system based on graph neural networks," *Knowledge-Based Systems*, vol. 258, p. 110030, 2023.
- [5] HCRL, Korea University, "Car-hacking dataset: In-vehicle CAN bus intrusion dataset," <https://ocslab.hksecurity.net/Datasets/car-hacking-dataset>, 2020, accessed: 2026-06-16.
- [6] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [7] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
- [8] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," *arXiv preprint arXiv:1903.02428*, 2019.
- [9] H. Yuan, H. Yu, S. Gui, and S. Ji, "Explainability in graph neural networks: A taxonomic survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 5787–5809, 2023.